LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

Disusun Oleh :

Nama	: Diki Hurrisyail
NIM	: (124140088)
Kelas Praktikum	: RA
Mata Kuliah	: Pemrograman Berorientasi Objek

Dosen Pengampu:

Arkham Zahri Rakhman, S.Kom., M.Eng.

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

INSTITUT TEKNOLOGI SUMATERA

2026

I. Tujuan Praktikum

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu memahami dan mengimplementasi konsep inheritance dalam Java. Secara lebih rinci Mahasiswa diharapkan mampu:

1. Menjelaskan konsep inheritance dalam OOP
2. Memahami hubungan superclass dan subclass
3. Menggunakan keyword `extends`
4. Mengakses atribut dan method dari class induk
5. Menggunakan keyword `super`
6. Menerapkan inheritance dalam studi kasus sederhana
7. Memahami manfaat penggunaan inheritance.

II. Alat dan Bahan

Pada praktikum ini alat dan bahan yang digunakan terdiri dari:

1. Laptop Windows 11: Sebagai perangkat utama untuk menjalankan, menulis, dan menguji program selama praktikum berlangsung.
2. JDK (Java Development Kit): Sebagai tools utama untuk mengembangkan, meng-compile, dan menjalankan program berbasis bahasa Java.
3. Visual Studio Code: Sebagai text editor atau IDE untuk menulis, mengedit, dan mengelola kode program Java dengan lebih mudah.
4. Modul Praktikum PBO BAB 4: Sebagai panduan dalam memahami materi serta langkah-langkah yang harus dilakukan selama praktikum.

III. Langkah Percobaan

Pada praktikum ini percobaan yang dilakukan seperti yang tertera pada subbab **4.16 Tugas Percobaan** di dalam modul. Berikut langkah percobaan yang dilakukan:

Percobaan 1:

Membuat class `Hewan` yang memiliki atribut `nama`, `umur`. Tambahkan method `makan()`, `tidur()`.

Kode Program:

```
class Hewan {
    String nama;
    int umur;

    void makan(){
        System.out.println(nama + " sedang makan");
    }

    void tidur(){
        System.out.println(nama + " sedang tidur");
    }
}
```

Percobaan 2:

Buat class `Kucing` yang merupakan turunan dari `Hewan`. Dengan atribut `ras`, method `mengeong()` dan `berburu()`. Lalu buat satu object `Kucing` dan tampilkan semua data serta methodnya.

Kode Program:

```
class Kucing extends Hewan{
    String ras;

    void mengeong() {
        System.out.println(nama + " mengeong...");
    }

    void berburu() {
        System.out.println(nama + " sedang berburu");
    }
}
```

```
public class MainHewan {
    public static void main(String[] args) {
        Kucing k = new Kucing();
        k.nama = "Humas";
        k.umur = 3;
        k.ras = "kamPUNK";

        System.out.println("KUCING");
        System.out.println("Nama: " + k.nama);
        System.out.println("Umur: " + k.umur);
        System.out.println("Ras: " + k.ras);

        k.makan();
        k.tidur();
        k.mengeong();
        k.berburu();
    }
}
```

Percobaan 3:

Membuat class `Anjing` yang merupakan turunan dari `Hewan`. Lalu menambahkan atribut `jenisGolongan` serta method `menggonggong()` dan `bermain()`.

Kode Program:

```
class Anjing extends Hewan {
    String jenisGolongan;

    void menggonggong() {
        System.out.println(nama + " menggonggong...");
    }

    void bermain() {
        System.out.println(nama + " sedang bermain");
    }
}
```

```
public class MainHewan {
    public static void main(String[] args) {
        Anjing a = new Anjing();
        a.nama = "Operasional";
        a.umur = 2;
        a.jenisGolongan = "kamPUNK";

        System.out.println("\nAnjing");
        System.out.println("Nama: " + a.nama);
        System.out.println("Umur: " + a.umur);
       System.out.println("Golongan: " + a.jenisGolongan);
```

```
        a.makan();
        a.tidur();
        a.menggonggong();
        a.bermain();
    }
}
```

Percobaan 4:

Mengubah class `Hewan`, `Kucing`, dan `Anjing` agar menggunakan constructor. Gunakan keyword `super` pada subclass

Kode Program:

```
class Hewan {
    String nama;
    int umur;

    Hewan(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }


    void makan(){
        System.out.println(nama + " sedang makan");
    }

    void tidur(){
        System.out.println(nama + " sedang tidur");
    }
}
```

```
class Kucing extends Hewan{
    String ras;

    public Kucing(String ras, String nama, int umur) {
        super(nama, umur);
        this.ras = ras;
    }


    void mengeong() {
        System.out.println(nama + " mengeong...");
    }

    void berburu() {
        System.out.println(nama + " sedang berburu");
    }
}
```

```
class Anjing extends Hewan {
    String jenisGolongan;

    public Anjing(String jenisGolongan, String nama, int
umur) {
        super(nama, umur);
        this.jenisGolongan = jenisGolongan;
    }

    void menggonggong() {
        System.out.println(nama + " menggonggong...");
    }

    void bermain() {
        System.out.println(nama + " sedang bermain");
    }
}
```

```
public class MainHewan {
    public static void main(String[] args) {
        Kucing k = new Kucing("kamPUNK", "Humas", 3);

        System.out.println("KUCING");
        System.out.println("Nama: " + k.nama);
        System.out.println("Umur: " + k.umur);
        System.out.println("Ras: " + k.ras);

        k.makan();
        k.tidur();
        k.mengeong();
        k.berburu();

         Anjing a = new Anjing("kamPUNK", "Operasional",
2);

        System.out.println("\nAnjing");
        System.out.println("Nama: " + a.nama);
        System.out.println("Umur: " + a.umur);
       System.out.println("Golongan: " + a.jenisGolongan);

        a.makan();
        a.tidur();
        a.menggonggong();
        a.bermain();
    }
}
```

Percobaan 5:

Buat class `GoldenRetriever` sebagai turunan class `Anjing`. Dengan method `berenang()` dan `mengambil()`. Lalu membuat satu object `GoldenRetriever` dan tunjukan bahwa object tersebut dapat menggunakan method dari `Hewan`, method dari `Anjing`, dan method miliknya sendiri

Kode Program:

```
class GoldenRetriever extends Anjing {
     GoldenRetriever (String jenisGolongan, String nama,
int umur){
        super(jenisGolongan, nama, umur);
    }

    void berenang() {
        System.out.println(nama + " sedang berenang");
    }

    void mengambil() {
        System.out.println(nama + " mengambil kayu");
    }
}
```

```
public class MainHewan {
    public static void main(String[] args) {
                        GoldenRetriever     gr     =     new
GoldenRetriever("Mamalia","Agit", 2);

        System.out.println("\nGolden Retriever");
        System.out.println("Nama: " + gr.nama);
        System.out.println("Umur: " + gr.umur);
                    System.out.println("Golongan:    "    +
gr.jenisGolongan);

        gr.makan();
        gr.tidur();

        gr.menggonggong();
        gr.bermain();

        gr.berenang();
        gr.mengambil();
    }
}
```

IV. Hasil Output Program

Percobaan 2:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan4 (main)
$ java -cp bin MainHewan
KUCING
Nama: Humas
Umur: 3
Ras: kamPUNK
Humas sedang makan
Humas sedang tidur
Humas mengeong...
Humas sedang berburu
```

Percobaan 3:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan4 (main)
$ java -cp bin MainHewan

Anjing
Nama: Operasional
Umur: 2
Golongan: kamPUNK
Operasional sedang makan
Operasional sedang tidur
Operasional menggonggong...
Operasional sedang bermain
```

Percobaan 4:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan4 (main)
$ java -cp bin MainHewan
KUCING
Nama: Humas
Umur: 1
Ras: kamPUNK
Humas sedang makan
Humas sedang tidur
Humas mengeong...
Humas sedang berburu

Anjing
Nama: Operasional
Umur: 2
Golongan: kamPUNK
Operasional sedang makan
Operasional sedang tidur
Operasional menggonggong...
Operasional sedang bermain
```

Percobaan 5:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan4 (main)
$ java -cp bin MainHewan

Golden Retriever
Nama: Agit
Umur: 2
Golongan: Mamalia
Agit sedang makan
Agit sedang tidur
Agit menggonggong...
Agit sedang bermain
Agit sedang berenang
Agit mengambil kayu
```

V. Analisis Program

Pada praktikum ini, class `Hewan` berperan sebagai superclass karena menjadi induk yang menyediakan atribut dasar yaitu `nama` dan `umur`, serta method `makan()` dan `tidur()`. Selanjutnya, class `Kucing` dan `Anjing` merupakan subclass yang mewarisi seluruh atribut dan method dari `Hewan`. Class `Kucing` menambahkan atribut `ras` serta method `mengeong()` dan `berburu()`, sedangkan class `Anjing` menambahkan atribut `jenisGolongan` serta method `menggonggong()` dan `bermain()`. Kemudian terdapat class `GoldenRetriever` yang merupakan turunan dari `Anjing`, sehingga membentuk pewarisan bertingkat.

Atribut dan method yang diwariskan dari `Hewan` ke subclass antara lain `nama`, `umur`, `makan()`, dan `tidur()`. Pewarisan ini dimungkinkan dengan penggunaan keyword `extends`, yang menunjukkan bahwa suatu class merupakan turunan dari class lain. Selain itu, penggunaan keyword `super` pada constructor subclass berfungsi untuk memanggil constructor milik superclass, sehingga atribut yang diwariskan dapat langsung diinisialisasi. Contohnya pada class `Kucing` dan `Anjing`, constructor menggunakan `super(nama, umur)` untuk mengisi nilai atribut dari class `Hewan`.

Perbedaan antara inheritance biasa dan inheritance bertingkat terletak pada jumlah level pewarisan. Inheritance biasa hanya melibatkan satu tingkat, yaitu subclass langsung mewarisi dari superclass (contohnya `Kucing` dan `Anjing` terhadap `Hewan`). Sedangkan inheritance bertingkat melibatkan lebih dari satu level, di mana sebuah subclass menjadi superclass bagi class lain (contohnya `GoldenRetriever` yang mewarisi dari `Anjing`, dan secara tidak langsung juga mewarisi dari `Hewan`). Dengan demikian, inheritance bertingkat memungkinkan pewarisan atribut dan method secara berantai dari level paling atas hingga level paling bawah.

VI. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, konsep inheritance dalam OOP dapat dipahami sebagai mekanisme pewarisan atribut dan method dari suatu class induk (superclass) ke class turunan (subclass). Melalui implementasi class `Hewan` sebagai superclass serta `Kucing`, `Anjing`, dan `GoldenRetriever` sebagai subclass, terlihat bahwa hubungan antar class membentuk hierarki yang jelas. Dengan demikian, mahasiswa dapat memahami bagaimana superclass menyediakan struktur dasar yang dapat dikembangkan lebih lanjut oleh subclass.

Penggunaan keyword `extends` berperan penting dalam mendefinisikan hubungan pewarisan antar class, sehingga subclass dapat langsung mengakses atribut dan method dari class induk tanpa perlu mendefinisikan ulang. Selain itu, keyword `super` digunakan untuk memanggil constructor dari superclass, sehingga proses inisialisasi atribut dapat dilakukan dengan lebih efisien dan terstruktur. Hal ini menunjukkan bahwa subclass tidak hanya mewarisi, tetapi juga dapat memperluas fungsionalitas dari superclass sesuai kebutuhan.

Melalui studi kasus sederhana yang melibatkan beberapa class turunan, dapat disimpulkan bahwa inheritance memberikan banyak manfaat, seperti mengurangi redundansi kode, meningkatkan keterbacaan program, serta mempermudah pengelolaan dan pengembangan sistem. Dengan adanya inheritance, struktur program menjadi lebih modular dan terorganisir, sehingga memudahkan dalam pemeliharaan maupun pengembangan lebih lanjut di masa depan.